1) SEPUTAR BIG DATA

- Inti besar
  Big Data adalah data dengan ukuran besar, kompleksitas tinggi, dan kecepatan tinggi sehingga tidak dapat diproses dengan metode konvensional. Fokus utama adalah prediction, bukan hanya inference.
- 5V Big Data
  Volume: ukuran data besar → masalah storage
  Velocity: data mengalir cepat → perlu real-time processing
  Variety: format beragam → sulit integrasi
  Veracity: kualitas data → noise dan bias
  Value: nilai bisnis → ekstraksi insight
  Intinya, 5V adalah karakteristik sekaligus tantangan analitik.
- Pipeline Big Data
  Alur wajib: Source → Storage → Processing → Modeling → Decision
  Data tidak bisa langsung ke model, harus melalui preprocessing. Kegagalan satu tahap mempengaruhi keseluruhan sistem.
- Batch vs Stream Processing
  Batch: data dikumpulkan lalu diproses, lebih stabil tapi lambat
  Stream: data diproses real-time, cepat tapi lebih kompleks
  Perbedaannya adalah trade-off antara latency dan completeness, bukan mana yang lebih baik.
- Paradigma Analitik
  Klasik: fokus pada inference, data kecil, metode statistik
  Big Data: fokus pada prediction, data besar, metode machine learning dan deep learning
  Big Data adalah perluasan statistik, bukan pengganti.
- Evaluasi Model
  Klasik: R-squared, AIC
  Big Data: error dan kemampuan generalisasi
  Untuk kasus forecasting digunakan MAPE: rata-rata persentase error, semakin kecil semakin baik.
- Workflow Big Data
  Ingestion → Processing → Modeling → Deployment → Feedback
  Model bersifat dinamis dan harus terus diperbarui.
- Hubungan Konsep
  Big Data → Processing → Machine Learning → Decision → Value
- Studi Kasus
  Contoh: prediksi permintaan makanan
  Masalah: bahan cepat rusak
  Solusi: model prediksi permintaan
  Langkah: preprocessing, feature engineering, modeling, evaluasi
  Jenis: supervised learning, time-based prediction
- Jebakan Konsep
  Big Data bukan hanya soal ukuran
  Batch vs stream adalah soal use case, bukan perbandingan mutlak
  Data lebih penting daripada model
  Pipeline tidak boleh dilewati
  Evaluasi harus sesuai jenis masalah
- Ringkasan Cepat
  Big Data adalah data besar, kompleks, dan cepat
  5V adalah karakteristik dan tantangan
  Pipeline bersifat end-to-end
  Big Data berfokus pada prediction
  Machine learning adalah inti analitik
  Evaluasi harus sesuai konteks
  Data lebih penting daripada model

2.1) SEPUTAR SUPERVISED LEARNING (REGRESI)

- Inti besar
  Supervised learning adalah model yang belajar dari data berlabel untuk memetakan X ke Y dengan tujuan prediksi, klasifikasi dan generalisasi, bukan sekadar menghafal.
  Unsupervised Learning adalah model belajar dari pola dalam data berupa X dengan tujuan menemukan struktur tersembunyi yang terbagi menjadi clustering (segmentasi) dan dimensionality reduction (PCA)
- Struktur masalah
  Input adalah fitur X dan output adalah target Y. Jenis Y menentukan model: jika Y kontinu maka regresi, jika Y kategorik maka klasifikasi.
- Regresi vs klasifikasi
  Regresi digunakan untuk output angka seperti harga atau tekanan darah, sedangkan klasifikasi digunakan untuk output kategori seperti lulus atau tidak. Kesalahan umum adalah mencampur keduanya.
- Regresi linier
  Model memprediksi nilai rata-rata Y sebagai fungsi linear dari X. Koefisien menunjukkan perubahan rata-rata Y ketika X naik satu unit dengan variabel lain konstan. Tujuan utama adalah estimasi hubungan dan interpretasi.
- Estimasi least squares
  Parameter diperoleh dengan meminimalkan jumlah kuadrat error. Metode ini sensitif terhadap outlier dan mengasumsikan hubungan linear pada parameter. Learning di sini adalah proses optimasi.
- Kelemahan regresi linier untuk klasifikasi
  Output tidak dibatasi antara 0 dan 1 sehingga tidak valid sebagai probabilitas, dan fungsi loss tidak sesuai untuk tujuan klasifikasi. Ini adalah alasan utama perlu model lain.
- Regresi logistik
  Digunakan untuk memodelkan probabilitas kejadian. Output berada dalam rentang 0 sampai 1 melalui fungsi sigmoid. Estimasi menggunakan maximum likelihood, bukan least squares.
- Interpretasi logistik
  Koefisien berada pada skala log-odds dan biasanya diubah menjadi odds ratio. Nilai lebih dari satu meningkatkan peluang, kurang dari satu menurunkan peluang. Ini sering menjadi sumber kebingungan.
- Decision threshold
  Klasifikasi ditentukan oleh threshold yang tidak selalu harus 0.5. Threshold mempengaruhi trade-off antara false positive dan false negative sehingga harus disesuaikan dengan konteks.
- Perbedaan utama linier vs logistik
  Regresi linier memodelkan nilai rata-rata dengan loss MSE, sedangkan regresi logistik memodelkan probabilitas dengan loss berbasis likelihood. Perbedaan utama terletak pada tujuan dan fungsi loss, bukan sekadar algoritma.
- Insight penting
  Supervised learning adalah kerangka hubungan X ke Y, bukan kumpulan algoritma. Loss function menentukan perilaku model. Regresi kuat untuk interpretasi, logistik lebih realistis untuk klasifikasi.
- Studi kasus
  Kasus regresi digunakan untuk memprediksi tekanan darah dari umur, IMT, dan aktivitas. Kasus klasifikasi digunakan untuk memprediksi kelulusan dari IPK, kehadiran, dan status bekerja. Interpretasi koefisien harus selalu dalam konteks.
- Jebakan ujian
  Sering salah membedakan regresi dan klasifikasi, salah interpretasi koefisien logistik, menganggap threshold selalu 0.5, dan tidak memahami bahwa loss function menentukan model.
- Ringkasan cepat
  Supervised learning memetakan X ke Y. Regresi untuk output kontinu, klasifikasi untuk kategori. Regresi linier menggunakan least squares, logistik menggunakan likelihood. Logistik menghasilkan probabilitas valid. Model ditentukan oleh tujuan, data, dan loss function. Regresi linear bisa dipakai klasifikasi (thresholding), walau tidak ideal.

2.2) SEPUTAR SUPERVISED LEARNING (JAWABAN KODING REGRESI)

- Mengapa regresi logistik lebih konsisten secara teoritis dibanding regresi linier untuk klasifikasi?
  Regresi logistik lebih konsisten karena memodelkan probabilitas dalam rentang $[0, 1]$ *melalui fungsi logit, sedangkan regresi linier dapat menghasilkan prediksi di luar batas probabilitas.*
- Bandingkan interpretasi β pada regresi linier dan logistik.
  β regresi linier merepresentasikan perubahan rata-rata nilai Y, sedangkan β regresi logistik merepresentasikan perubahan log-odds (atau odds ratio setelah eksponensiasi).
- Dalam konteks apa regresi linier tetap dapat digunakan pada data biner?
  Regresi linier masih bisa digunakan pada data biner sebagai pendekatan kasar atau baseline ketika fokus pada prediksi sederhana, bukan interpretasi probabilistik.
- Mengapa fungsi loss kuadrat tidak selaras dengan tujuan klasifikasi?
  Loss kuadrat tidak cocok untuk klasifikasi karena tidak memodelkan probabilitas dan tidak memberikan penalti yang sesuai terhadap kesalahan prediksi kelas.
- Apa implikasi penggunaan threshold terhadap interpretasi statistik model?
  Threshold menentukan keputusan klasifikasi, sehingga mengubah trade-off antara false positive dan false negative serta memengaruhi interpretasi hasil model.
- Interpretasikan secara substantif odds ratio untuk tiap variabel.
  Odds ratio >1 → meningkatkan peluang sedangkan Odds ratio <1 → menurunkan peluang
- Jelaskan peran fungsi logit dalam menjaga konsistensi probabilitas.
  Logit memastikan output model konsisten sebagai probabilitas yang valid.
- Berikan interpretasi substantif koefisien variabel.
  Koefisien = perubahan rata-rata Y saat X naik 1
- Diskusikan perbedaan antara signifikansi statistik dan relevansi klinis.
  Signifikan ≠ selalu penting secara klinis
- Dalam kondisi apa interpretasi kausal dapat dibenarkan?
- *Kausal hanya jika tidak ada bias & desain valid, tanpa kondisi ini hasil hanya asosiasi (korelasi), bukan sebab-akibat.*
- Jelaskan interpretasi βluas dalam konteks kasus prediksi harga rumah berdasarkan luas bangunan dan jumlah kamar sebagai estimasi marginal E(Y|X). *β_luas merepresentasikan perubahan rata-rata harga rumah untuk setiap kenaikan 1 unit luas bangunan, dengan variabel lain (jumlah kamar) konstan.*
- Dalam konteks apa intercept memiliki makna substantif?
  Intercept bermakna substantif hanya jika kondisi semua variabel = 0 masuk akal dalam konteks nyata; jika tidak, hanya berfungsi sebagai baseline matematis.
- Mengapa asumsi linearitas tidak berarti hubungan data harus membentuk garis lurus?
  Linearitas berarti linear pada parameter (β), bukan harus garis lurus pada data, sehingga hubungan bisa tetap kompleks melalui kombinasi variabel.
- Inferensi vs Prediksi:
  Inferensi bertujuan memahami hubungan dan efek variabel (β) untuk penjelasan (misal: pengaruh IMT terhadap tekanan darah), sedangkan prediksi berfokus pada akurasi hasil (misal: memprediksi kelulusan), sehingga pemilihan model dan interpretasi koefisien disesuaikan dengan tujuan (penjelasan vs performa).
- Fungsi Loss dan Estimasi Parameter:
  Regresi linier menggunakan MSE (least squares) yang sensitif terhadap outlier, sedangkan regresi logistik menggunakan log-likelihood Bernoulli (cross-entropy) yang sesuai untuk probabilitas, sehingga estimasi parameter logistik lebih konsisten untuk klasifikasi dan lebih robust terhadap kesalahan probabilistik.
- Interpretasi Probabilistik & Kebijakan:
  Koefisien log-odds sulit dipahami non-teknis, sehingga perlu ditransformasikan menjadi odds ratio atau probabilitas agar lebih mudah dikomunikasikan dalam pengambilan keputusan.
- Threshold & Risiko Kesalahan
  Threshold 0.5 tidak selalu optimal karena harus disesuaikan dengan trade-off FP dan FN, misalnya pada diagnosis penyakit lebih penting meminimalkan FN (recall tinggi) dibanding sekadar akurasi.
- Keterbatasan Model & Alternatif
  Regresi logistik tidak memadai ketika hubungan antar variabel non-linear atau kompleks, sehingga diperlukan model alternatif seperti tree-based atau boosting yang mampu menangkap interaksi dan pola non-linear.

3.1) SEPUTAR METRIK EVALUASI & PEMILIHAN MODEL

- Inti besar
  Evaluasi model bertujuan menilai kualitas model dan memastikan generalisasi, bukan sekadar akurasi.
- Training vs test error
  Training error dihitung pada data latih, sedangkan test error pada data baru. Model yang baik harus memiliki performa baik pada test, bukan hanya training.
- Overfitting dan underfitting
  Underfitting terjadi saat model terlalu sederhana, overfitting saat model terlalu kompleks dan menghafal data. Tujuan adalah mencari keseimbangan optimal.
- Bias-variance trade-off
  Bias tinggi menyebabkan underfitting, variance tinggi menyebabkan overfitting. Big data dapat mengurangi variance tetapi tidak otomatis menghilangkan bias.
- Metrik regresi
  MSE error kuadrat, menghukum error besar dan sensitif terhadap outlier.
  RMSE adalah akar MSE sehingga mudah diinterpretasi.
  MAE rata-rata error absolut, lebih robust terhadap outlier.
  R squared mengukur proporsi variasi yang dijelaskan model.
  Adjusted R squared mengoreksi jumlah variabel.
  MAPE mengukur error persentase tetapi tidak valid jika nilai aktual nol.
- Metrik klasifikasi
  Cross entropy mengukur seberapa jauh probabilitas prediksi dari label asli. Dipakai di: Logistic regression & Neural network
  Accuracy mengukur proporsi prediksi benar tetapi menyesatkan pada data tidak seimbang. Rumus: [TP + TN/Total Data]
  Precision penting ketika false positive mahal (meminimalkan FP). Rumus: [TP/TP + FP]
  Recall penting ketika false negative mahal (meminimalkan FN). Rumus: [TP/TP + FN]
  F1 score menyeimbangkan precision dan recall. Rumus: [2 x precision x recall/precision + recall]
  ROC-AUC mengukur performa model secara keseluruhan tanpa tergantung threshold.
  AUC (Nilai Akhirnya)
  0.5 → tebak random (kayak lempar koin)
  1.0 → sempurna
  Precision-recall curve lebih relevan untuk data tidak seimbang.
- Validasi model
  Evaluasi pada data training bersifat bias optimistik karena model sudah melihat data. Validasi diperlukan untuk estimasi performa nyata.
  Hold-out sederhana tetapi kurang stabil.
  K-fold cross validation lebih stabil karena menggunakan berbagai subset data.
  Stratified CV menjaga proporsi kelas dan penting untuk data imbalance.
- Pemilihan model
  Model terbaik bukan yang paling kompleks tetapi yang memiliki kinerja baik, stabil, dan sesuai konteks.
  Pertimbangan meliputi metrik, stabilitas, kompleksitas, interpretabilitas, dan biaya komputasi.
  AIC dan BIC menyeimbangkan goodness of fit dan kompleksitas tetapi tidak selalu optimal untuk prediksi.
- Outlier
  Outlier dapat menyebabkan distorsi pada hasil model machine learning. Pada umumnya, cara pengangannya adalah dengan transformasi (log), scaling (robust scaler), deteksi (IQR/Z-score) atau membuangnya.
- Konsep Overfitting Dan Underfitting
  Overfitting terjadi jika model terlalu kompleks yang berimplikasi pada model menjadi hafal data training dan jelek pada saat data testing. Sedangkan underfitting terjadi jika model terlalu sederhana yang berimplikasi pada performa yang jelek pada data train maupun test.
  Cara mengatasi: overfitting → regularization, cross-validation, pruning. underfitting → tambah kompleksitas model
- Studi kasus
  Pada churn dengan data tidak seimbang dan biaya false negative tinggi, metrik yang tepat adalah recall atau F1.
  Pada forecasting dengan banyak outlier, MAE lebih sesuai dibanding MSE.
- Pola pikir kunci
  Pemilihan metrik harus berdasarkan tujuan, jenis data, dan biaya kesalahan, bukan hanya akurasi.
- Perbedaan konsep
  Evaluasi mengukur performa, validasi mengestimasi generalisasi, dan model selection memilih model terbaik berdasarkan keduanya.
- Jebakan ujian
  Menggunakan accuracy pada data imbalance, tidak memisahkan data training dan testing, tidak memahami trade-off bias-variance, dan memilih model terlalu kompleks.
- Ringkasan cepat
  Evaluasi bukan hanya akurasi. Training tidak sama dengan test. Overfitting adalah risiko utama. Gunakan metrik sesuai masalah. Cross validation penting untuk estimasi stabil. Model terbaik adalah yang tepat guna, bukan paling kompleks.

3.2) SEPUTAR METRIK EVALUASI & PEMILIHAN MODEL (JAWABAN KODING MATRIKS EVALUASI)

Evaluasi model bukan sekadar memilih nilai metrik terkecil, melainkan memahami konteks inferensial, risiko kesalahan, dan tujuan analisis.

- Mengapa model dengan MSE training lebih kecil belum tentu lebih baik?
  MSE training kecil ≠ model terbaik. MSE training kecil bisa menandakan overfitting sehingga performa pada data baru justru buruk.
- Jelaskan hubungan kompleksitas model dengan varians estimasi.
  Semakin kompleks model → varians estimasi meningkat → model lebih sensitif terhadap perubahan data.
- Dalam konteks kebijakan kesehatan, metrik mana lebih relevan?
  Recall lebih relevan karena meminimalkan false negative (tidak melewatkan pasien sakit).
- Mengapa evaluasi pada data training menghasilkan estimasi bias terhadap error sebenarnya?
  Evaluasi pada data training bias optimistik karena model sudah "melihat" data tersebut.
- Bandingkan interpretasi AUC dengan accuracy.
  Accuracy bergantung pada threshold, sedangkan AUC mengukur performa model secara keseluruhan tanpa tergantung threshold.
- Kapan model dengan AIC lebih kecil belum tentu terbaik untuk prediksi?
  AIC kecil ≠ terbaik untuk prediksi. AIC fokus pada keseimbangan fit dan kompleksitas untuk inferensi, bukan performa prediksi pada data baru.
- Jelaskan hubungan evaluasi model dengan trade-off bias–variance.
  Evaluasi model bertujuan menemukan keseimbangan antara bias (underfitting) dan variance (overfitting).
- Feature Selection & PCA.
  Feature Selection: Haps fitur dan Pilih fitur penting serta Output tetap interpretabel
  PCA: Transformasi fitur dengan tujuan mengecilkan dimensi namun tidak bisa interpretasi langsung

4) SEPUTAR METODE BERBASIS POHON

- Inti besar
  Tree digunakan untuk mengatasi keterbatasan model linear dalam menangkap hubungan nonlinear dan interaksi kompleks antar variabel. Tree tidak butuh scaling
- Intuisi dasar
  Tree membagi ruang data menjadi beberapa region menggunakan aturan if-then sehingga prediksi dibuat berdasarkan kelompok data yang homogen.
- CART (Classification and Regression Trees)
  CART adalah model non-parametrik berbasis data yang membangun pohon dengan struktur root, node, dan leaf. Model bekerja dengan binary split untuk mempartisi data.
- Regression tree
  Digunakan untuk memprediksi nilai kontinu dengan tujuan meminimalkan error dalam setiap region. Kriteria split menggunakan penurunan RSS sehingga tiap kelompok memiliki variasi kecil.
- Classification tree
  Digunakan untuk memprediksi kelas dengan tujuan memaksimalkan kemurnian kelompok. Kriteria split menggunakan Gini atau entropy untuk meminimalkan impurity.
- Karakteristik tree
  Tree mampu menangkap nonlinearitas dan interaksi secara otomatis serta tidak memerlukan scaling. Namun tree memiliki varians tinggi, tidak stabil, dan mudah overfitting.
- Overfitting pada tree
  Terjadi ketika tree terlalu dalam sehingga model sangat cocok pada data training tetapi buruk pada data baru. Tree memiliki bias rendah tetapi variance tinggi sehingga rentan overfitting.
- Pruning
  Pruning adalah proses menyederhanakan tree untuk mengurangi overfitting dengan mengurangi jumlah leaf. Parameter kompleksitas mengontrol trade-off antara error dan ukuran tree. Pruning berfungsi sebagai regularisasi pada tree.
- Ensemble methods
  Bagging menggunakan bootstrap sampling dan averaging untuk mengurangi variance.
  Random Forest menambahkan randomisasi fitur untuk mengurangi korelasi antar tree sehingga lebih stabil.
  Boosting melatih model secara berurutan dengan fokus pada error sehingga meningkatkan performa dan menurunkan bias.
- Perbandingan model
  CART memiliki variance tinggi dan mudah overfit.
  Random Forest lebih stabil karena mengurangi variance.
  Boosting memiliki performa tinggi karena mengurangi bias dan memodelkan error.
- Praktikum
  Kontrol kompleksitas dilakukan dengan membatasi kedalaman tree. Feature importance digunakan untuk mengetahui variabel yang berpengaruh. Perbandingan model dilakukan berdasarkan error dan stabilitas.
- Insight penting
  Tree adalah model fleksibel tetapi rawan overfitting. Pruning hanya solusi parsial, sedangkan ensemble adalah solusi utama. Banyak tree lebih baik daripada satu tree.
- Jebakan ujian
  Menganggap tree selalu terbaik, tidak membatasi kedalaman, tidak memahami bias-variance, tidak membedakan bagging dan boosting, serta salah interpretasi feature importance.
- Ringkasan cepat
  Tree membagi data menjadi region homogen. Regresi menggunakan RSS, klasifikasi menggunakan Gini atau entropy. Masalah utama adalah overfitting. Solusi adalah pruning dan ensemble. Random Forest stabil, boosting memberikan performa tinggi.
- Bias-Variance

Model	*Karakter*
Tree tunggal	*overfitting*
Random Forest	*reduce variance*
Boosting	*reduce bias*

5) SEPUTAR ENSMBLE METHODS

- Inti besar
  Ensemble methods menggabungkan banyak model untuk mengatasi kelemahan tree terutama high variance dan meningkatkan akurasi.
- Masalah utama CART
  Tree memiliki variance tinggi, tidak stabil, dan mudah overfitting sehingga perubahan kecil pada data dapat mengubah struktur model secara drastis.
- Konsep ensemble
  Menggabungkan banyak model lemah menjadi model kuat dengan tujuan utama menurunkan variance dan meningkatkan performa.
- Prinsip penting
  Ensemble efektif jika model tidak terlalu mirip. Korelasi antar model harus rendah agar penurunan variance maksimal.
- Bagging
  Menggunakan bootstrap sampling dan averaging untuk mengurangi variance.
  Model dilatih secara paralel.
  Bias tetap, variance menurun.
  Out-of-bag error dapat digunakan sebagai estimasi performa tanpa cross validation.
- Random Forest
  Pengembangan bagging dengan menambahkan random subset fitur pada setiap split.
  Tujuannya mengurangi korelasi antar tree sehingga model lebih stabil.
  Random Forest bekerja karena memaksa model berbeda.
- Boosting
  Melatih model secara berurutan dengan fokus pada error model sebelumnya.
  Mengurangi bias dan meningkatkan performa.
  Menggunakan model sederhana sebagai weak learner.
  Learning rate mengontrol kecepatan dan kompleksitas model.
- Perbedaan utama bagging vs boosting
  Bagging bekerja secara paralel dan fokus pada penurunan variance.
  Boosting bekerja secara sequential dan fokus pada penurunan bias serta memperbaiki error.
- BART
  Model ensemble berbasis Bayesian yang menggabungkan banyak tree kecil dan memberikan estimasi uncertainty.
  Lebih cocok untuk inferensi tetapi lebih berat secara komputasi.
- Evolusi metode
  CART fleksibel tetapi tidak stabil.
  Bagging menstabilkan model.
  Random Forest mengurangi korelasi dan lebih stabil.
  Boosting meningkatkan akurasi dengan mengurangi bias.
  BART menambahkan pendekatan probabilistik.
- Perbandingan model
  CART memiliki variance tinggi.
  Bagging mengurangi variance.
  Random Forest lebih stabil karena decorrelation.
  Boosting memberikan performa tinggi dengan risiko overfitting jika tidak dikontrol.
- Insight penting
  Ensemble bukan sekadar banyak model tetapi kombinasi model yang beragam.
  Random Forest unggul dalam stabilitas.
  Boosting unggul dalam akurasi.
  Diversitas model adalah kunci.
- Jebakan ujian
  Menganggap semua ensemble sama, tidak memahami perbedaan bias dan variance, tidak membedakan paralel dan sequential, serta mengabaikan korelasi antar model.
  Ringkasan cepat
  Ensemble menggabungkan banyak model untuk meningkatkan performa.
  Bagging mengurangi variance, Random Forest mengurangi korelasi, Boosting mengurangi bias.
  Model yang beragam menghasilkan performa lebih baik daripada model tunggal.

Model	Bias	Variance	Interpretasi
CART	Rendah	Tinggi	Tinggi
Random Forest	Rendah	Rendah	Sedang
Boosting	Sangat Rendah	Sedang	Rendah
BART	Rendah	Rendah	Rendah

6) SEPUTAR NAIVE BAYES & KNN

- Inti besar
  Naive Bayes dan KNN adalah dua pendekatan klasifikasi yang berbeda, yaitu probabilistic dan instance-based, dengan cara belajar yang sangat berbeda.
- Perbedaan konsep
  Naive Bayes membangun model probabilitas berdasarkan distribusi data, sedangkan KNN tidak membangun model dan hanya menggunakan data untuk menghitung kemiripan saat prediksi.
- Naive Bayes
  Model berbasis Teorema Bayes yang menghitung probabilitas kelas berdasarkan prior dan likelihood. Prediksi dilakukan dengan memilih kelas dengan probabilitas terbesar.
- Asumsi Naive Bayes
  Fitur diasumsikan saling independen terhadap kelas. Asumsi ini sering tidak realistis tetapi tetap bekerja baik dalam banyak kasus.
- Komponen utama
  Prior adalah probabilitas awal kelas, likelihood adalah probabilitas fitur diberikan kelas, dan posterior adalah hasil akhir untuk prediksi.
- Jenis Naive Bayes
  Gaussian untuk data kontinu dengan asumsi normal, multinomial untuk data teks atau frekuensi.
- Zero probability problem
  Jika suatu probabilitas nol maka hasil keseluruhan menjadi nol. Solusinya adalah Laplace smoothing untuk menghindari nilai nol.
- Kelebihan dan kelemahan Naive Bayes
  Cepat, sederhana, dan cocok untuk data berdimensi tinggi, tetapi tidak mampu menangkap interaksi antar fitur.
- KNN
  Model berbasis jarak yang mengklasifikasikan data berdasarkan k tetangga terdekat. Tidak memiliki proses training sehingga disebut lazy learning.
- Mekanisme KNN
  Hitung jarak ke semua data, pilih k terdekat, lalu lakukan voting untuk menentukan kelas. KNN = bisa klasifikasi & regresi, tapi sering dipakai klasifikasi
- Pemilihan k
  k kecil menghasilkan model kompleks dengan variance tinggi (overfitting), sedangkan k besar menghasilkan model sederhana dengan bias tinggi (underfitting). Pemilihan k biasanya menggunakan cross validation.
- Normalisasi
  KNN sangat sensitif terhadap skala fitur sehingga normalisasi atau standardisasi wajib dilakukan agar fitur tidak mendominasi. Normalisasi juga bertujuan agar semua fitur memiliki skala yang sama serta mengurangi varians/bias.
- Kelebihan dan kelemahan KNN
  Mudah dipahami dan fleksibel untuk pola nonlinear, tetapi lambat saat prediksi, sensitif terhadap noise, dan tidak cocok untuk data berdimensi tinggi.
- Curse of dimensionality
  Pada dimensi tinggi, jarak antar data menjadi tidak bermakna sehingga performa KNN menurun drastis.
- Perbandingan Naive Bayes dan KNN
  Naive Bayes cepat dan cocok untuk data besar serta berdimensi tinggi, sedangkan KNN lebih cocok untuk data kecil dan pola lokal tetapi tidak skalabel.
- Insight penting
  Naive Bayes adalah model global berbasis probabilitas, sedangkan KNN adalah model lokal berbasis kemiripan. Keduanya memiliki kelebihan pada kondisi data yang berbeda.
- Jebakan ujian
  Mengabaikan asumsi independensi pada Naive Bayes, tidak menggunakan smoothing, tidak melakukan normalisasi pada KNN, memilih k sembarangan, dan menggunakan KNN pada data besar atau berdimensi tinggi.
- Ringkasan cepat
  Naive Bayes menggunakan probabilitas dan cepat untuk data besar, KNN menggunakan jarak dan cocok untuk data kecil. KNN membutuhkan normalisasi dan sensitif terhadap dimensi tinggi. Tidak ada model terbaik, pemilihan tergantung data dan tujuan.

7) SEPUTAR SVM

- Inti besar
  SVM adalah model klasifikasi yang mencari batas pemisah terbaik dengan margin maksimum untuk menghasilkan generalisasi yang baik.
- Intuisi dasar
  Banyak garis dapat memisahkan data tetapi SVM memilih hyperplane dengan margin terbesar karena lebih stabil terhadap data baru.
- Hyperplane dan margin
  Hyperplane adalah batas pemisah antar kelas, sedangkan margin adalah jarak antara hyperplane dan titik terdekat. Margin besar menghasilkan model yang lebih robust.
- Support vector
  Hanya titik yang berada di dekat margin yang menentukan model sehingga tidak semua data berpengaruh.
- Hard margin
  Digunakan jika data dapat dipisahkan sempurna. Tidak mengizinkan error tetapi sensitif terhadap outlier dan tidak realistis pada data nyata.
- Soft margin
  Mengizinkan error untuk menangani data yang tidak separable. Parameter C mengontrol trade-off antara margin dan error.
- Parameter C
  Nilai besar membuat model fokus pada error sehingga margin kecil dan berisiko overfitting. Nilai kecil membuat margin besar tetapi dapat underfitting. Parameter C pada SVM mengontrol trade-off antara margin besar dan error kecil.
- Hinge loss
  Loss function yang hanya menghukum prediksi yang salah atau terlalu dekat dengan batas, sehingga tidak semua error diperlakukan sama.
- Nonlinear SVM dan kernel
  Jika data tidak linear, kernel digunakan untuk memetakan data ke dimensi lebih tinggi sehingga dapat dipisahkan secara linear.
- Jenis kernel
  Linear cocok untuk data besar dan sederhana. Polynomial menangkap interaksi antar fitur. RBF paling fleksibel dan umum digunakan.
- Parameter gamma
  Gamma besar menghasilkan boundary kompleks dan berisiko overfitting. Gamma kecil menghasilkan boundary lebih halus dan berisiko underfitting. Gamma pada SVM kernel RBF mengontrol seberapa jauh pengaruh satu data terhadap data lain.
- Support vector regression
  Versi regresi dari SVM yang mengabaikan error kecil dalam suatu batas toleransi dan hanya fokus pada error besar.
- Parameter penting
  C mengontrol regularisasi, gamma mengontrol kompleksitas, dan epsilon mengontrol toleransi error pada regresi.
- Kapan digunakan
  Cocok untuk data berdimensi tinggi dan boundary kompleks. Kurang cocok untuk data sangat besar atau ketika interpretasi model sangat dibutuhkan.
- Insight penting
  SVM adalah model berbasis margin yang hanya bergantung pada support vector. Kernel berfungsi sebagai feature engineering implisit.
- Jebakan ujian

Salah memahami peran margin, tertukar antara C dan gamma, mengira semua data penting, dan tidak memahami fungsi kernel.

- Ringkasan cepat
  SVM mencari boundary dengan margin maksimum. C mengontrol error dan gamma mengontrol kompleksitas. Kernel membuat model mampu menangani data nonlinear. Hanya support vector yang menentukan model.

Support Vector Regression (SVR)

Motivasi dan Ide Utama SVR:

- *SVR adalah bentuk penerapan Support Vector Machine (SVM) yang digunakan untuk tugas regresi, yaitu memprediksi nilai kontinu, dengan tetap mempertahankan konsep margin.*
- *Ide utamanya adalah mencari fungsi yang dapat mendekati data dengan batasan toleransi error tertentu.*
- *Dalam SVR, tidak semua error diberikan hukuman (penalti); model ini hanya memfokuskan perhatian pada error yang berukuran besar.*

Fungsi dan Persamaan Matematis SVR

- *Fungsi Model: Dirumuskan sebagai f(x)=wT x + b yang sifatnya linear di ruang fitur sama seperti SVM.*
- *Loss Function: SVR menggunakan Hinge-like Loss yang dirumuskan sebagai max(0, | yi - f(xi)| - ϵ) Tidak penalti jika error kecil, Penalti linear jika melewati ϵ*
- *Objective Function: Model berupaya meminimalkan fungsi min(12||w||2+C(i+i*)) dimana i dan i* adalah slack variables. C: penalti error*

Peran Support Vectors pada SVR

- *Hanya titik di luar -tube yang berpengaruh titik ini support vectors.*
- *Hanya titik-titik (support vectors) inilah yang berkontribusi dan menentukan bentuk fungsi model.*
- *Data di dalam tube sama sekali tidak berkontribusi pada model.*

Pengaruh Parameter Penting

- *Parameter : Menentukan lebar dari tube. Semakin besar nilainya, model menjadi semakin toleran terhadap error.*
- *Parameter C: Menentukan besaran penalti untuk error. Semakin besar nilai C, model akan semakin ketat dalam mengikuti data.*
- *Parameter Y: Parameter ini berfungsi mengontrol kelengkungan model Khusus untuk kernel RBF*

Ringkasan

- *SVR adalah versi regresi dari SVM*
- *Menggunakan ϵ-insensitive loss*
- *Fokus pada error besar*
- *Mendukung kernel trick*

Alur dan Konsep Utama SVM

- *Hyperplane: Garis pemisah antar kelas.*
- *Maximum Margin: untuk memaksimalkan jarak aman antar kelas.*
- *Soft Margin: Menggunakan slack variables untuk memberikan toleransi terhadap error.*
- *Kernel Trick: untuk menangani data nonlinear.*
- *Kapan Menggunakan SVM?*
- *Dataset dimensi tinggi dan jumlah fitur besar*
- *Boundary kompleks*
- *Ukuran data tidak terlalu besar*

Pemilihan Jenis Kernel

- *Kernel Linear: Digunakan saat data berukuran besar dengan dimensi fitur tinggi, contohnya pada pemrosesan teks.*
- *Kernel Polynomial: Digunakan apabila interaksi antar fitur di dalam data memiliki peran penting.*
- *Kernel RBF: Merupakan pilihan bawaan (default choice) yang paling disarankan untuk menangani data yang kompleks atau nonlinear.*

101) TRIVIA/PERTANYAAN

- PERTANYAAN:
  Sebuah model klasifikasi penyakit memiliki:
  Accuracy tinggi (95%) & Recall rendah (60%)
  Analisis:
  Apakah model ini baik? Apa masalahnya? Apa solusi yang bisa dilakukan?
  JAWAB:
  Masalah: model gagal deteksi positif (FN tinggi)
  Solusi: threshold tuning, class balancing (SMOTE/oversampling), fokus recall / gunakan recall/F1, ATAU ubah loss function
- PERTANYAAN:
  Jelaskan perbedaan L1 vs L2 Regularization dari segi: Cara kerja, Dampak pada model, dan Kapan digunakan.
  JAWAB:
  L1 mengecilkan koefisien menjadi nol sedangkan L2 mengecilkan koefisien secara merata. Oleh karena itu, L1 berguna untuk pemilihan fitur, karena dapat menghilangkan variabel apa pun yang terkait dengan koefisien yang mendekati nol. L2, di sisi lain, berguna ketika memiliki fitur kolinear/kodependen/multikolinearitas.
- PERTANYAAN:
  Mengapa cross-validation lebih disarankan dibandingkan train-test split dalam banyak kasus?
  JAWAB:
  CV digunakan untuk menguji model berulang kali menggunakan bagian data yang berbeda untuk pelatihan dan pengujian ketika data terbatas. Tujuan utamanya adalah untuk mengukur performa model secara lebih stabil serta mengurangi bias split tunggal.
- PERTANYAAN:
  Sebuah model mengalami overfitting. Jelaskan minimal 3 cara untuk mengatasinya.
  JAWAB:
  Regularization (L1/L2), Cross-validation, Pruning (tree), Kurangi fitur, Tambah data, Dropout (DL)
- PERTANYAAN:
  Diberikan kasus: dataset memiliki Banyak fitur (100+), Banyak fitur tidak relevan, Performa model buruk
  Analisis: Apa penyebabnya? Teknik apa yang sebaiknya digunakan? Jelaskan alurnya secara singkat
  JAWAB:
  Penyebab: fitur tidak relevan, noise tinggi, overfitting
  Solusi: feature selection (L1, filter method), PCA (dimensionality reduction)
  Alur: cleaning → feature selection/PCA → modeling
- PERTANYAAN:
  Sebuah model deteksi penyakit memiliki: Accuracy: 96% Precision: 85% Recall: 50%
  Pertanyaan: Apakah model ini layak digunakan? Jelaskan.
  Apa masalah utamanya? Berikan minimal 3 solusi berbeda (bukan 1 jenis saja)
  JAWAB:
  Tidak layak → karena FN tinggi (pasien sakit lolos)
  Solusi: threshold tuning, class balancing (SMOTE/oversampling), gunakan recall/F1, cost-sensitive learning (penalize FN lebih besar)
- PERTANYAAN:
  Dataset memiliki: 200 fitur, Banyak fitur saling berkorelasi, Model sulit diinterpretasi
  Pertanyaan: Lebih cocok PCA atau Feature Selection? Jelaskan alas an Apa trade-off masing-masing?
  JAWAB:
  Jika fokus interpretasi → Feature Selection
  Jika fitur saling berkorelasi tinggi → PCA lebih kuat
  Trade-off:
  FS: interpretabel, tapi tidak tangani multikolinearitas optimal
  PCA: hilang interpretasi, tapi kompres informasi
  harus selalu bandingkan dua sisi
- PERTANYAAN:
  Model KNN digunakan pada data dengan 100 fitur, performa buruk.
  Analisis: Apa penyebab utama? Kenapa KNN sangat terpengaruh kondisi ini? Solusi apa yang bisa dilakukan (minimal 2)
  JAWAB:
  Penyebab: sparsity tinggi, distance concentration
  Solusi: PCA, feature selection, scaling
- PERTANYAAN:
  Model mengalami overfitting parah.
  Pertanyaan: Jelaskan bagaimana L1 dan L2 membantu? Kapan memilih L1 vs L2? Tambahkan minimal 2 solusi lain selain regularization.
  JAWAB:
  Solus: L1 → sparsity, L2 → shrinkage
  Tambahan: cross-validation, kurangi fitur, tambah data, pruning (tree)
- PERTANYAAN:
  Sebuah model SVM menunjukkan: Training accuracy tinggi namun Testing accuracy rendah
  Analisis: Apa yang terjadi? Bagaimana pengaruh parameter C dan gamma? Bagaimana cara tuning untuk memperbaiki model?
  JAWAB:
  Terjadi overfitting. C besar yang membuat margin kecil dan penalti error tinggi, kemudian gamma besar yang membuat model menjadi kompleks.
  Solusi: kecilkan C (lebih toleran error), kecilkan gamma (lebih smooth), gunakan cross-validation (GridSearch)

KODING REGRESI

```
library(tidyverse)

#BAGIAN 1: REGRESI LINIER SEDERHANA
#Kasus Harga Rumah
# Membuat data frame manual
luas <- c(50, 60, 70, 80)
kamar <- c(2, 2, 3, 3)
harga <- c(300, 350, 400, 450)
data <- data.frame(luas, kamar, harga)
# Estimasi model OLS
model_lin <- lm(harga ~ luas + kamar, data = data)
summary(model_lin)

#BAGIAN 2: REGRESI LINIER UNTUK DATA BINER
#Simulasi Data Biner (Halaman 3)
set.seed(123)
X <- seq(-3, 3, length.out = 50)
# Membuat Y biner (0 atau 1) dengan noise
Y <- ifelse(X + rnorm(50, 0, 0.8) > 0, 1, 0)
data_bin <- data.frame(X, Y)
# Memaksa model linier pada data biner
model_lin_bin <- lm(Y ~ X, data = data_bin)
summary(model_lin_bin)
# Plot hasil regresi linier pada data biner
ggplot(data_bin, aes(x = X, y = Y)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  labs(title = "Regresi Linier pada Data Biner",
       x = "X",
       y = "Y (Biner)") +
  theme_minimal()


#BAGIAN 3: REGRESI LOGISTIK
#Estimasi Maximum Likelihood (Halaman 4)
model_log <- glm(Y ~ X, data = data_bin,
                 family = binomial(link = "logit"))
summary(model_log)
# Mendapatkan Odds Ratio
exp(coef(model_log))
# Plot hasil regresi logistik pada data biner
ggplot(data_bin, aes(x = X, y = Y)) +
  geom_point() +
  stat_smooth(method = "glm", method.args = list(family = "binomial"), se = FALSE,
color = "red") +
  labs(title = "Regresi Logistik pada Data Biner",
       x = "X",
       y = "Probabilitas Y=1") +
  theme_minimal()

#BAGIAN 4: STUDI KASUS KESEHATAN (Regresi Linier)
#Simulasi Data Kesehatan (Halaman 5-6)
set.seed(2026)
n <- 120
umur <- round(runif(n, 25, 70))
IMT <- round(rnorm(n, 25, 4), 1)
aktivitas <- round(runif(n, 0, 10), 1)
# Membuat Y dengan rumus linier + error normal
tekanan_darah <- 90 + 0.6 * umur + 1.2 * IMT - 1.5 * aktivitas + rnorm(n, 0, 8)
data_kesehatan <- data.frame(umur, IMT, aktivitas, tekanan_darah)
# Modeling
model_kes <- lm(tekanan_darah ~ umur + IMT + aktivitas, data = data_kesehatan)
summary(model_kes)
# Plot hasil regresi linier pada data kesehatan
ggplot(data_kesehatan, aes(x = umur, y = tekanan_darah)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "green") +
  labs(title = "Regresi Linier pada Data Kesehatan",
       x = "Umur",
       y = "Tekanan Darah") +
  theme_minimal()

#BAGIAN 5: STUDI KASUS KELULUSAN (Regresi Logistik)
#Simulasi Data Kelulusan
set.seed(2026)
n <- 150
IPK <- round(runif(n, 2.2, 4.0), 2)
kehadiran <- round(runif(n, 60, 100), 1)
bekerja <- rbinom(n, 1, 0.35)
# Fungsi Logit (Linear Predictor)
linpred <- -15 + 4*IPK + 0.08*kehadiran - 1.2*bekerja
# Fungsi Sigmoid (Probabilitas)
prob_lulus <- 1 / (1 + exp(-linpred))
lulus <- rbinom(n, 1, prob_lulus)
data_kelulusan <- data.frame(IPK, kehadiran, bekerja, lulus)
# Modeling
model_lulus <- glm(lulus ~ IPK + kehadiran + bekerja,
                   data = data_kelulusan,
                   family = binomial(link = "logit"))
summary(model_lulus)
exp(coef(model_lulus)) # Odds Ratio
# Plot hasil regresi logistik pada data kelulusan
ggplot(data_kelulusan, aes(x = IPK, y = lulus)) +
  geom_point() +
  stat_smooth(method = "glm", method.args = list(family = "binomial"), se = FALSE,
color = "purple") +
  labs(title = "Regresi Logistik pada Data Kelulusan",
       x = "IPK",
       y = "Probabilitas Lulus") +
  theme_minimal()
```

KODING MATRIKS EVALUASI

```
library(tidyverse)

# --- 1.1 Data Simulasi Realistis (Kesehatan) ---
set.seed(2026)
n <- 200
# Membangkitkan variabel independen
umur <- round(runif(n, 25, 70))
IMT <- round(rnorm(n, 25, 4), 1)
aktivitas <- round(runif(n, 0, 10), 1)
# Membangkitkan variabel dependen (Tekanan Darah)
# Catatan: Tanda operasi disesuaikan berdasarkan logika Python di dokumen (hal 2)
tekanan_darah <- 90 + 0.6 * umur + 1.2 * IMT - 1.5 * aktivitas + rnorm(n, 0, 8)
data_kes <- data.frame(
  umur, IMT, aktivitas, tekanan_darah
)
# --- 1.2 Train-Test Split ---
set.seed(1)
idx <- sample(1:n, size = 0.7 * n)
train <- data_kes[idx, ]
test <- data_kes[-idx, ]
# --- 1.3 Estimasi Model ---
# Model 1: Linear biasa
model1 <- lm(tekanan_darah ~ umur + IMT + aktivitas, data = train)
# Model 2: Menambahkan suku kuadratik (Non-linear pada umur)
model2 <- lm(tekanan_darah ~ umur + IMT + aktivitas + I(umur^2), data = train)
# --- 1.4 Metrik Evaluasi (MSE & RMSE) ---
pred1 <- predict(model1, test)
pred2 <- predict(model2, test)
# Hitung MSE
mse1 <- mean((test$tekanan_darah - pred1)^2)
mse2 <- mean((test$tekanan_darah - pred2)^2)
print(paste("MSE Model 1:", mse1))
print(paste("MSE Model 2:", mse2))
# Hitung RMSE
rmse1 <- sqrt(mse1)
rmse2 <- sqrt(mse2)
print(paste("RMSE Model 1:", rmse1))
print(paste("RMSE Model 2:", rmse2))

# --- 2.1 Data Simulasi Kelulusan Mahasiswa ---
set.seed(2026)
n <- 250
IPK <- round(runif(n, 2.2, 4.0), 2)
kehadiran <- round(runif(n, 60, 100), 1)
bekerja <- rbinom(n, 1, 0.35)
# Fungsi linear prediktor (Log-odds)
# Catatan: Menggunakan koefisien dari dokumen (-15, +4, +0.08, -1.2)
linpred <- -15 + 4 * IPK + 0.08 * kehadiran - 1.2 * bekerja
# Mengubah ke probabilitas (Sigmoid)
prob_lulus <- 1 / (1 + exp(-linpred))
# Generate label kelas (0/1) berdasarkan probabilitas
lulus <- rbinom(n, 1, prob_lulus)
data_kel <- data.frame(IPK, kehadiran, bekerja, lulus)
# --- 2.2 Model Logistik ---
model_log <- glm(lulus ~ IPK + kehadiran + bekerja,
                 data = data_kel,
                 family = binomial)
# Prediksi probabilitas pada data yang sama (untuk evaluasi training)
prob_pred <- predict(model_log, type = "response")
# --- 2.3 Confusion Matrix ---
threshold <- 0.5
pred_class <- ifelse(prob_pred > threshold, 1, 0)
# Membuat tabel kontingensi (Confusion Matrix)
conf_matrix <- table(Predicted = pred_class, Actual = data_kel$lulus)
print(conf_matrix)
# --- 2.4 Metrik Klasifikasi Manual ---
# TP: True Positive, TN: True Negative, FP: False Positive, FN: False Negative
TP <- conf_matrix[2, 2]
TN <- conf_matrix[1, 1]
FP <- conf_matrix[2, 1]
FN <- conf_matrix[1, 2]
accuracy <- (TP + TN) / sum(conf_matrix)
precision <- TP / (TP + FP)
recall <- TP / (TP + FN)
print(paste("Accuracy:", accuracy))
print(paste("Precision:", precision))
print(paste("Recall:", recall))
# --- 2.5 ROC dan AUC ---
library(pROC)
roc_obj <- roc(data_kel$lulus, prob_pred)
plot(roc_obj, main="ROC Curve")
auc_value <- auc(roc_obj)
print(paste("AUC:", auc_value))
# --- 3.1 K-Fold Cross Validation ---
library(caret)

train_control <- trainControl(
method = "cv",
number = 5
)
cv_model <- train(
tekanan_darah ~ umur + IMT + aktivitas,
data = data_kes,
method = "lm",
trControl = train_control
)
cv_model
```

KODING TREE-BASED METHODS

```
#(1) Persiapan
#(1.1) Instalasi dan Library
install.packages(c(
"tree",
"rpart",
"rpart.plot",
"randomForest ",
"gbm",
"caret",
"BART",
"ISLR2"
))

library(tree)
library(rpart)
library(rpart.plot)
library(randomForest)
library(gbm)
library(caret)
library(BART)
library(ISLR2)

#(2) Dataset
#Digunakan dua dataset:
#• Boston (regresi)
#• Carseats (klasifikasi)
data(Boston)
data(Carseats)

#(3) Regression Tree (Boston Housing)
#(3.1) Split Data
set.seed(123)
train_index <- createDataPartition(Boston$medv, p = 0.7, list = FALSE)
train <- Boston[train_index, ]
test <- Boston[-train_index, ]
#(3.2) Membangun CART
#Model regresi tree merepresentasikan fungsi:
# f(x)=m=1Mcm1(x Rm)
tree_model <- tree(medv ~ ., data = train)
summary(tree_model)
plot(tree_model)
text(tree_model, pretty = 0)
#(3.3) Pruning
cv_tree <- cv.tree(tree_model)
plot(cv_tree$size, cv_tree$dev, type = "b")
best_size <- cv_tree$size[which.min(cv_tree$dev)]
pruned_tree <- prune.tree(tree_model, best = best_size)
#(3.4) Evaluasi
pred_tree <- predict(pruned_tree, test)
rmse_tree <- sqrt(mean((pred_tree - test$medv)^2)
)
rmse_tree

#(4) Bagging
#Bagging menggunakan bootstrap sampling dan averaging.
bag_model <- randomForest(medv ~ ., data = train,
mtry = ncol ( train ) -1,
importance = TRUE
)
pred_bag <- predict(bag_model, test)
rmse_bag <- sqrt(mean((pred_bag - test$medv)^2))
rmse_bag
#OOB error:
bag_model$mse[length(bag_model$mse)]

#(5) Random Forest
rf_model <- randomForest(medv ~ ., data = train, importance = TRUE)
pred_rf <- predict(rf_model, test)
rmse_rf <- sqrt(mean((pred_rf - test$medv) ^2))
rmse_rf
#(5.1) Variable Importance
importance(rf_model)
varImpPlot(rf_model)

#(6) Boosting (Gradient Boosting)
#Boosting membangun model aditif:
FM(x)=m=1Mfm(x)
boost_model <- gbm(medv ~ ., data = train,
distribution = " gaussian ",
n.trees = 2000,
interaction.depth = 3,
shrinkage = 0.01,
cv. folds = 5
)
best_iter <- gbm.perf(boost_model, method = "cv")
pred_boost <- predict(boost_model, test, n.trees = best_iter)
rmse_boost <- sqrt(mean((pred_boost - test$medv)^2))
rmse_boost

#(7) Bayesian Additive Regression Trees (BART)
#BART merepresentasikan:
# Y = j=1mg(x; Tj,Mj) +
x_train <- as.matrix(train[, -which(names(train) == "medv")])
y_train <- train$medv
x_test <- as.matrix(test[, -which(names(test) == "medv")])
bart_model <- wbart(x_train, y_train)
pred_bart <- predict(bart_model, x_test)
yhat_bart <- colMeans(pred_bart)
rmse_bart <- sqrt(mean((yhat_bart - test$medv)^2))
rmse_bart
#(7.1) Interval Kredibel
lower <- apply(pred_bart, 2, quantile, 0.025)
upper <- apply(pred_bart, 2, quantile, 0.975)

#(8) Perbandingan Model
results <- data.frame(
Model = c("CART", "Bagging", "Random Forest", "Boosting", "BART"),
RMSE = c(rmse_tree, rmse_bag, rmse_rf, rmse_boost, rmse_bart))
results

#(9) Classification (Carseats)
Carseats$High <- ifelse(Carseats$Sales > 8, "Yes", "No")
Carseats$High <- as.factor(Carseats$High)
Carseats$Sales <- NULL
set.seed(123)
train_ index <- createDataPartition(Carseats$High, p = 0.7, list = FALSE)
train <- Carseats[train_index, ]
test <- Carseats[-train_index, ]
rf_class <- randomForest(High ~ ., data = train)
pred_class <- predict(rf_class, test)
confusionMatrix(pred_class, test$High)
```

INTERPRETASI

1. Regression Tree (CART)
Model membagi data menjadi beberapa region dengan nilai prediksi konstan (piecewise constant).
Hasil summary biasanya menunjukkan jumlah node kecil → model sederhana.
RMSE relatif besar → indikasi underfitting / high variance belum dikontrol.
Insight: tree tunggal mudah overfit tetapi performa generalisasi sering kalah dari ensemble.
2. Pruning
Cross-validation memilih ukuran tree optimal untuk menyeimbangkan bias dan variance.
Pruned tree biasanya lebih kecil dan RMSE test lebih baik dibanding tree awal.
Insight: pruning = regularisasi untuk mengurangi overfitting.
3. Bagging
Menggunakan banyak tree dengan bootstrap sampling → averaging.
RMSE lebih kecil dibanding CART → variance berkurang.
OOB error mendekati error test → estimasi performa cukup akurat tanpa CV.
Insight: bagging efektif untuk model dengan variance tinggi.
4. Random Forest
Menambahkan random subset fitur → mengurangi korelasi antar tree.
RMSE biasanya lebih kecil dari bagging → model lebih stabil dan generalisasi lebih baik.
Variable importance menunjukkan fitur paling berpengaruh terhadap prediksi.
Insight: Random Forest = versi bagging yang lebih kuat karena decorrelation.
5. Boosting (GBM)
Model dibangun secara sequential dengan fokus pada error sebelumnya.
RMSE biasanya paling kecil atau mendekati terbaik → bias berkurang.
Parameter penting:
depth → kompleksitas
shrinkage → learning rate
Insight: boosting unggul dalam akurasi tetapi sensitif terhadap tuning dan noise.
6. BART
Ensemble berbasis Bayesian → menghasilkan distribusi prediksi, bukan hanya titik estimasi.
RMSE kompetitif dengan boosting.
Interval kredibel memberikan ketidakpastian prediksi.
Insight: BART unggul untuk inferensi dan uncertainty, bukan hanya prediksi.
7. Perbandingan Model (Expected Pattern)
Urutan umum performa:
CART > Bagging > Random Forest > Boosting ≈ BART (terbaik)
CART: paling lemah (variance tinggi)
Bagging: lebih stabil
RF: lebih baik karena decorrelation
Boosting: akurasi tinggi
BART: akurat + ada uncertainty
8. Classification (Random Forest - Carseats)
Confusion matrix menunjukkan:
Accuracy tinggi
Precision dan recall bergantung distribusi kelas
Insight penting:
Jika data imbalance → accuracy bisa menyesatkan
Perhatikan recall jika false negative mahal
9. Insight Besar (WAJIB HAFAL)
Tree tunggal → variance tinggi → overfitting
Bagging → reduce variance
Random Forest → reduce correlation → lebih stabil
Boosting → reduce bias → akurasi tinggi
BART → probabilistik → ada uncertainty
10. Kesimpulan
Ensemble methods secara konsisten mengungguli single tree karena mampu mengurangi variance dan bias, dengan Random Forest memberikan stabilitas tinggi dan Boosting memberikan akurasi terbaik, sedangkan BART menambahkan keunggulan dalam estimasi ketidakpastian.